



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ
ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ
СПЕЦИАЛНОСТ КОМПЮТЪРНО И СОФТУЕРНО
ИНЖЕНЕРСТВО

Студент: Валери Ивайлов Райков

Фак. №: 121222139

Група: 42Б

Дисциплина: Паралелно Програмиране

Преподавател: гл. ас. Петър Маринов

Курсова работа на тема М2-13:

Thread вместо Task. Реализирайте едно от упражненията ПП Модул 2 като не използвате класа Task, а класа Thread и други класове от System.Threading

Дата на предаване: 01/11/2025г.

Паралелно търсене на клиенти с използване на Thread в WPF приложение

1. Резюме

Проектът представя разработка на настолно WPF приложение на езика C#, което извършва паралелно търсене на клиенти в колекция по идентификатор (ID), използвайки многопоточност чрез класа Thread. Реализирана е симулация на бавен достъп до данни, а търсенето се извършва едновременно в няколко нишки. Приложението визуализира резултатите в графичен интерфейс и демонстрира основните принципи на работа с нишки в .NET. Направено е и кратко сравнение с алтернативен подход – асинхронно програмиране чрез Task, което е реализирано по време на упражнението.

2. Съдържание

Паралелно търсене на клиенти с използване на Thread в WPF приложение	2
1. Резюме	2
3. Въведение	3
3.1 Обща информация.....	3
3.2 Цели на проекта.....	3
3.3 Обхват на работа	3
4. Теоретична основа.....	3
4.1 Преглед на литературата	3
4.2 Алгоритми и концепции	3
5. Методология	4
5.1 Избор на технологии.....	4
5.2 Процес на разработка.....	4
5.3 Паралелизация на алгоритъма	4
6. Реализация.....	4
6.1 Архитектура на проекта.....	4
6.2 Основни фрагменти от кода	5
6.3 Инсталация и стартиране.....	7
7. Резултати	7
7.1 Тестови данни.....	7
7.2 Анализ на производителността.....	7
7.3 Визуализация	7
8. Обсъждане.....	7
8.1 Интерпретация на резултатите.....	7
8.2 Ограничения	8

9.	Заклучение.....	8
10.	Библиография.....	8
11.	Приложения:.....	8

3. Въведение

3.1 Обща информация

В съвременните приложения често се налага едновременно изпълнение на множество операции, без да се блокира потребителският интерфейс. В .NET това може да се постигне чрез използване на **нишки (Thread)**, които позволяват паралелно изпълнение на независими задачи. Този проект демонстрира използването на Thread за едновременно търсене на клиенти в локална колекция. Приложението е разработено с WPF и цели да илюстрира ръчната работа с нишки, синхронизация и достъп до UI потока.

3.2 Цели на проекта

- Да се реализира паралелно търсене на клиенти чрез Thread.
- Да се гарантира безопасен достъп до споделени ресурси чрез lock.
- Да се осигури коректно взаимодействие между нишките и графичния интерфейс.
- Да се сравни подходът с алтернативния модел на Task.

3.3 Обхват на работа

Проектът обхваща само локално търсене в списък от клиенти. Не се разглеждат бази данни, мрежови заявки или външни източници, а се фокусира върху разглеждането на едновременно търсене с използването на класа Thread и сравнение с реализация на същото задание, използваща класа Task (представена по време на упражнението).

4. Теоретична основа

4.1.Преглед на литературата

Класът Thread от пространството имена System.Threading е един от най-старите и фундаментални механизми за многопоточност в .NET. Всеки Thread представлява отделен поток на изпълнение в процеса. Нишките споделят обща памет, което ги прави ефективни, но изисква синхронизация при достъп до споделени обекти.

4.2 Алгоритми и концепции

- **Thread.Start()** – стартира изпълнение на нов поток.
- **Thread.Join()** – блокира текущата нишка, докато другата приключи.
- **lock(object)** – механизъм за синхронизиран достъп до общи ресурси.
- **Dispatcher.Invoke()** – използва се във WPF, за да се извършват UI операции от фонове нишки.

Сравнение:

При Task тези механизми са скрити от разработчика — синхронизацията и управлението на нишки се извършват автоматично от **ThreadPool** и **Task Scheduler**, което улеснява разработчика, но не му дава толкова голяма гъвкавост и контрол над програмата.

5. Методология

5.1 Избор на технологии

- **Програмен Език:** C# (.NET 9.0)
- **Интерфейс:** Windows Presentation Foundation (WPF)
- **Библиотеки:** System.Threading, System.Linq

5.2 Процес на разработка

1. Създаване на клас **Customer** с основни свойства.
2. Създаване на хранилище **CustomerRepository** с фиктивни данни.
3. Имплементация на метод за търсене с Thread.Sleep() (симулация на забавяне – произволно забавяне чрез използване на класа Random).
4. Реализация на паралелно търсене с няколко нишки.
5. Изчакване на всички нишки чрез Join().
6. Обновяване на UI с Dispatcher.Invoke().

5.3 Паралелизация на алгоритъма

За всяко ID се създава отделна нишка, която изпълнява следните стъпки:

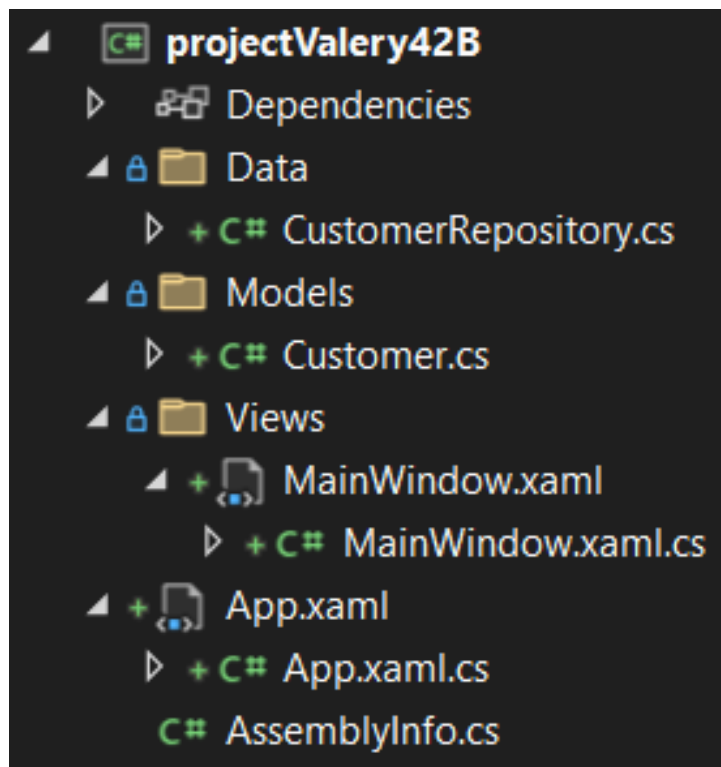
- Изчаква случайно време (500–5000 ms).
- Търси клиент в колекцията.
- При успешен резултат добавя клиента в общ списък, защитен с lock.
- След приключване на всички нишки, резултатите се визуализират.

6. Реализация

6.1 Архитектура на проекта

Въпреки че обема на проекта не е голям и файловете, които съдържа, не са много на брой, те са структурирани следвайки добрите принципи на разработка на софтуерни решения в WPF, а именно:

- **Директория Models:** съдържа структура на клиентите.
- **Директория Data:** съдържа логиката за търсене и симулация на забавяне.
- **Директория View:** представя потребителския интерфейс на приложението.



6.2 Основни фрагменти от кода

- Търсене на клиент в отделна нишка:

```
public Customer? FindCustomerById(int id)
{
    int delay = _rand.Next(500, 5000);
    Thread.Sleep(delay);
    return _customers.FirstOrDefault(c => c.Id == id);
}
```

- Паралелно търсене с Thread:

```
public Customer[] FindCustomers(IEnumerable<int> ids)
{
    var foundCustomers = new List<Customer>();
    var threads = new List<Thread>();
    var lockObj = new object();

    foreach (var id in ids)
    {
        var thread = new Thread(() =>
        {
            var result = FindCustomerById(id);
            if (result != null)
            {
                lockObj;
                foundCustomers.Add(result);
            }
        });
        threads.Add(thread);
        thread.Start();
    }

    foreach (var thread in threads)
    {
        thread.Join();
    }

    return foundCustomers.ToArray();
}
```

```

        {
            lock (lockObj)
            {
                foundCustomers.Add(result);
            }
        }
    });
    thread.Start();
    threads.Add(thread);
}

foreach (var t in threads)
    t.Join();

return foundCustomers.ToArray();
}

```

- Извикване от MainWindow.xaml.cs:

```

Thread thread = new Thread(() =>
{
    try
    {
        var found = _repository.FindCustomers(ids);

        Dispatcher.Invoke(() =>
        {
            ResultList.Items.Clear();
            if (found.Length == 0)
                ResultList.Items.Add("Няма намерени  
клиенти.");
            else
            {
                foreach (var c in found)
                    ResultList.Items.Add(c);
            }
        });
    }
    catch (Exception ex)
    {
        Dispatcher.Invoke(() =>
        {
            MessageBox.Show($"Грешка: {ex.Message}");
        });
    }
    finally
    {
        Dispatcher.Invoke(() =>
        {
            ButtonSearch.IsEnabled = true;
        });
    }
});

```

```
        });  
    }  
});
```

6.3 Инсталация и стартиране

1. Отворете проекта в Visual Studio.
2. Компилирайте (Ctrl+Shift+B).
3. Стартирайте приложението (F5).
4. Въведете ID-та, разделени с интервали, и натиснете бутона „Търси клиенти“.

7. Резултати

7.1 Тестови данни

В хранилището са налични 10 фиктивни клиента с ID от 1 до 10. Използвани са различни комбинации от ID-та за търсене.

7.2 Анализ на производителността

Паралелното търсене с Thread показва значително по-добро време от последователно търсене. UI остава отзивчив, тъй като основният поток (основната нишка) не се блокира, а това гарантира много по-добър User Experience.

Сравнение:

Реализацията с **Task** има подобно време, но по-малко ръчен код. При **Thread** имаме повече контрол, но и по-голяма отговорност за управление на нишките, съответно шанса да допуснем грешка е по-голям и трябва достатъчно добре да знаем какво правим. **Task** е по-съвременният, по-ефективен и по-удобен за реални приложения. Въпреки това, разбирането на **Thread** е важно за по-ниско ниво контрол и оптимизация.

7.3 Визуализация

Приложението визуализира намерените клиенти в ListBox, като всеки ред съдържа ID, име, адрес и брой поръчки.

8. Обсъждане

8.1 Интерпретация на резултатите

Реализацията с Thread е ефективна при малък брой паралелни операции и показва отлично разбиране на ниско ниво работа с нишки. Недостатъкът е, че управлението им става сложно при по-големи системи, тъй като създаването на прекалено много нишки може да натовари системата.

8.2 Ограничения

- Липсва автоматично управление на нишките (както при решението с Task).
- Не е реализирана възможност за прекъсване на текущи търсения.
- При голям брой заявки е възможна прекомерна употреба на системни ресурси.
- Task би бил по-подходящ за мащабируемост и лесна отмяна.

9. Заключение

Проектът демонстрира успешна реализация на паралелно търсене чрез Thread в WPF среда. Показано е как чрез няколко нишки може да се ускори обработката и да се запази отзивчивостта на интерфейса. Въпреки това, за по-големи или реални приложения се препоръчва използването на Task, който предоставя по-лесно управление, автоматична оптимизация чрез ThreadPool и поддръжка на async/await. По-високото ниво на абстракция улеснява работата на програмиста и затова този подход е идеален за по-сложни разработки.

10. Библиография

1. Microsoft Docs – System.Threading Namespace
2. Microsoft Learn – Threads and Threading in .NET
3. Stack Overflow – Thread vs Task performance in C#
4. YouTube tutorials – C# Multithreading

11. Приложения:

В тази част от документацията ще направим последно сравнение между двете решения на заданието: чрез използване на Thread (текущия вариант) и чрез използване на Task (вариантът на решението от упражнението):

Характеристика	Thread	Task
Управление	Ръчно	Автоматично
Сложност	По-голяма	По-ниска
Контрол	Пълен контрол	Ограничен (но по-безопасен)
Изпълнение	Отделни нишки	ThreadPool
Подходящ за	CPU-интензивни задачи	I/O и асинхронни операции
Кодова четимост	По-труден за поддръжка	По-интуитивен
Синхронизация	Чрез lock	Вграден механизъм

Също така ще покажем как изглежда интерфейса на приложението, което ще демонстрираме и по време на защитата на курсовата работа:

Търсене на клиенти

Въведете ID-та (разделени с интервал):

1 3 5

Търси клиенти

Изход

Списък с клиенти:

1: Иван Иванов, ул. Първа 1, Поръчки: 5
2: Мария Петрова, ул. Втора 2, Поръчки: 3
3: Георги Георгиев, ул. Трета 3, Поръчки: 8
4: Елена Димитрова, ул. Четвърта 4, Поръчки: 2
5: Николай Николов, ул. Пета 5, Поръчки: 7
6: Стамат Пешов, ул. Шеста 6, Поръчки: 4
7: Марин Митев, ул. Седма 7, Поръчки: 2
8: Виктория Георгиева, ул. Осма 8, Поръчки: 1
9: Лили Филипова, ул. Девета 9, Поръчки: 6
10: Никола Михов, ул. Десета 10, Поръчки: 5

© 2025, Паралелно търсене на клиенти – демо проект

Търсене на клиенти

Въведете ID-та (разделени с интервал):

1 2

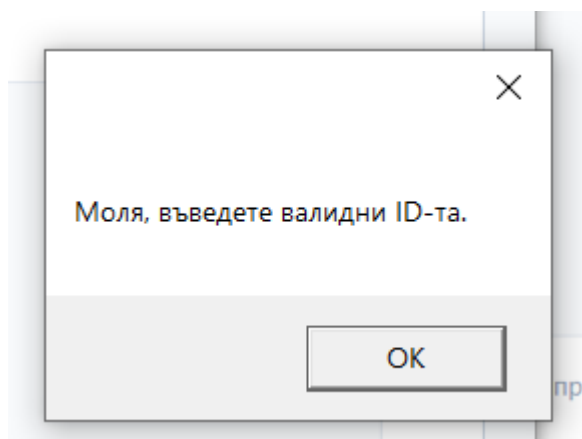
Търси клиенти

Изход

Списък с клиенти:

1: Иван Иванов, ул. Първа 1, Поръчки: 5
2: Мария Петрова, ул. Втора 2, Поръчки: 3

© 2025, Паралелно търсене на клиенти – демо проект



Търсене на клиенти

Въведете ID-та (разделени с интервал):

100

Търси клиенти

Изход

Списък с клиенти:

Няма намерени клиенти.